

1. Java Sort

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

Input Format

The first line of input contains an integer , representing the total number of students. The next lines contains a list of student information in the following structure:

ID Name CGPA

Constraints

The name contains only lowercase English letters. The contains only integer numbers without leading zeros. The *CGPA* will contain, at most, 2 digits after the decimal point.

Output Format

After rearranging the students according to the above rules, print the first name of each student on a separate line.

Sample Input

5

33 Rumpa 3.68

85 Ashis 3.85

56 Samiha 3.75

19 Samara 3.75

22 Fahim 3.76

Sample Output

Ashis

Fahim

Samara

Samiha

Rumpa

PROGRAM

```
import java.util.*;
```

```
class Student {
```

```
    private int id;
```

```
    private String name;
```

```
    private double cgpa;
```

```
    public Student(int id, String name, double cgpa) {
```

```
        this.id = id;
```

```
this.name = name;
```

```
this.cgpa = cgpa;
```

```
}
```

```
public int getId() {
```

```
    return id;
```

```
}
```

```
public String getName() {
```

```
    return name;
```

```
}
```

```
public double getCgpa() {
```

```
    return cgpa;
```

```
}
```

```
}
```

```
class Checker implements Comparator<Student> {
```

```
    @Override
```

```
    public int compare(Student s1, Student s2) {
```

```
        // 1. CGPA - decreasing order
```

```
        if (s1.getCgpa() != s2.getCgpa()) {
```

```
            return Double.compare(s2.getCgpa(), s1.getCgpa());
```

```
        }
```

```
// 2. Name - alphabetical order
```

```
int nameCompare = s1.getName().compareTo(s2.getName());
```

```
if (nameCompare != 0) {
```

```
    return nameCompare;
```

```
}
```

```
// 3. ID - increasing order
```

```
return Integer.compare(s1.getId(), s2.getId());
```

```
}
```

```
}
```

```
public class Solution {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        List<Student> students = new ArrayList<>();
```

```
        for (int i = 0; i < n; i++) {
```

```
            int id = sc.nextInt();
```

```
            String name = sc.next();
```

```
            double cgpa = sc.nextDouble();
```

```
            students.add(new Student(id, name, cgpa));
```

```
        }
```

```
Collections.sort(students, new Checker());
```

```
for (Student student : students) {
```

```
    System.out.println(student.getName());
```

```
}
```

```
sc.close();
```

```
}
```

```
}
```

OUTPUT

The screenshot shows a web browser window with the URL <https://www.hackerrank.com/challenges/java-sort/problem>. The page displays a 'Congratulations!' message, indicating that the user has passed the sample test cases. Below the message, there is a section for 'Sample Test case 0' which shows the input and output of the program.

Input (stdin)

```
5
33 Rumpa 3.68
85 Ashis 3.85
56 Samiha 3.75
19 Samara 3.75
22 Fahim 3.76
```

Your Output (stdout)

```
Ashis
Fahim
Samara
```

The browser's taskbar at the bottom shows the system time as 10:14 on 12-09-2026, with a battery level of 41% and a temperature of 30°C.